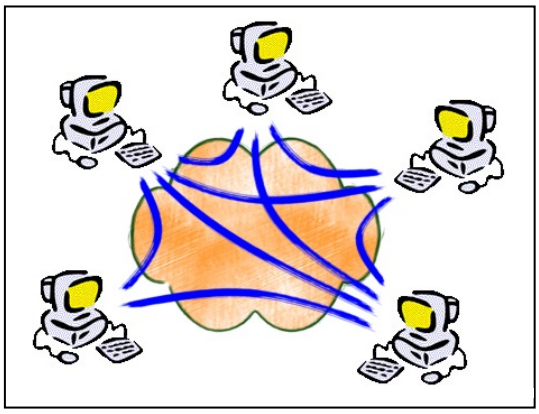
Prof. Dr. Erwin Mayer

Fakultät für Elektrotechnik
Medizin und Informatik

Praktikum Verteilte Systeme

Versuch 1 (2 Labortermine)

Peer-to-Peer Computing



Was Sie in diesem Versuch lernen:

- Vertrautwerden mit einer einfachen verteilten Anwendung
- Realisierung eines einfachen Peer-to-Peer Multi-Player Computer Games
- Aufbau eines Peer-to-Peer Gruppenmanagements
- Diskussion von Entwurfsentscheidungen für verteilte Systeme
- Absprache von Schnittstellen im Team und verteilte Komponentenentwicklung
- Testen von verteilten Anwendungen

Voraussetzung zur Teilnahme: Pre-Lab Aufgaben komplett durchgeführt (mit Nachweis) !

```
public class UdpReceiver {
    public static void main(String[] args) {
        // 1. Definition von Port und Puffergröße
        int port = 9876;
        byte[] buffer = new byte[1024]; // Puffer für max. 1024 Bytes

        System.out.println("Warte auf UDP-Nachricht auf Port " + port + "...");

        // 2. Socket an den Port binden (Try-with-Resources sorgt für automatisches Schließen)
        try (DatagramSocket socket = new DatagramSocket(port)) {

            // 3. Vorbereiten des DatagramPackets für den Empfang
            DatagramPacket packet = new DatagramPacket(buffer, buffer.length);

            // 4. Blockierendes Empfangen der Nachricht
            // Das Programm pausiert hier, bis ein Paket ankommt
            socket.receive(packet);

            // 5. Auswerten der empfangenen Daten
            String receivedMessage = new String(packet.getData(), 0, packet.getLength());

            // Metadaten des Absenders auslesen
            String senderAddress = packet.getAddress().getHostAddress();
            int senderPort = packet.getPort();

            // Ausgabe des Ergebnisses
            System.out.println("\n--- Nachricht empfangen! ---");
            System.out.println("Inhalt: " + receivedMessage);
            System.out.println("Von IP : " + senderAddress);
            System.out.println("Von Port: " + senderPort);

        } catch (Exception e) {
            System.err.println("Fehler beim UDP-Empfang: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

1 PRE-LAB-Fragen

Name	Matrikelnummer	Semester

1. Was ist der prinzipielle Unterschied zwischen verteilten Anwendungen, die nach dem Peer-to-Peer-Prinzip arbeiten und solchen, die das Client-Server-Prinzip anwenden.

Client-Server:

- server = Dienstanbieter
- client = Dienstnehmer
- klare Rollenverteilung

P2P:
- jeder kann Client und Server sein
- jeder kann mit jedem Kommunizieren

2. Angenommen ein Prozess muss gleichzeitig auf eine Benutzereingabe von der Konsole und auf das Eintreffen einer Nachricht warten. Wie realisieren Sie dies?

Multithreading

3. Wie realisieren Sie das Empfangen einer einzelnen UDP-Nachricht über eine Socket-Schnittstelle mit der Programmiersprache Ihrer Wahl (Java, Python, C, C++, C#). Bitte auch alle vorbereitenden Schritte.

4. Wie erzeugen Sie eine Nachricht mit einem Integer und einem String als Inhalt mit einer Programmiersprache Ihrer Wahl (Java, Python, C/C#/C++). Verwenden Sie, wo möglich, JSON-basierte Nachrichten.

```
public record Message(int id, String name){}

public static void main(String[] args){

   ObjectMapper mapper = new ObjectMapper();
   Message msg = new Message(1, "bla");
   String json = mapper.writeValueAsString(msg);
   System.out.println(json);
}
```

5. Wie erzeugen Sie eine Nachricht mit einem Integer und einer Liste mit Strings als Inhalt mit einer Programmiersprache Ihrer Wahl (Java, Python, C/C#/C++). Verwenden Sie, wo möglich, JSON-basierte Nachrichten.

```
public record Message(int id, List<String> names){}

public static void main(String[] args){

   ObjectMapper mapper = new ObjectMapper();
   List <String> names = List.off("bla", "blabla");
   Message msg = new Message(1, names);
   String json = mapper.writeValueAsString(msg);
   System.out.println(json);
}
```

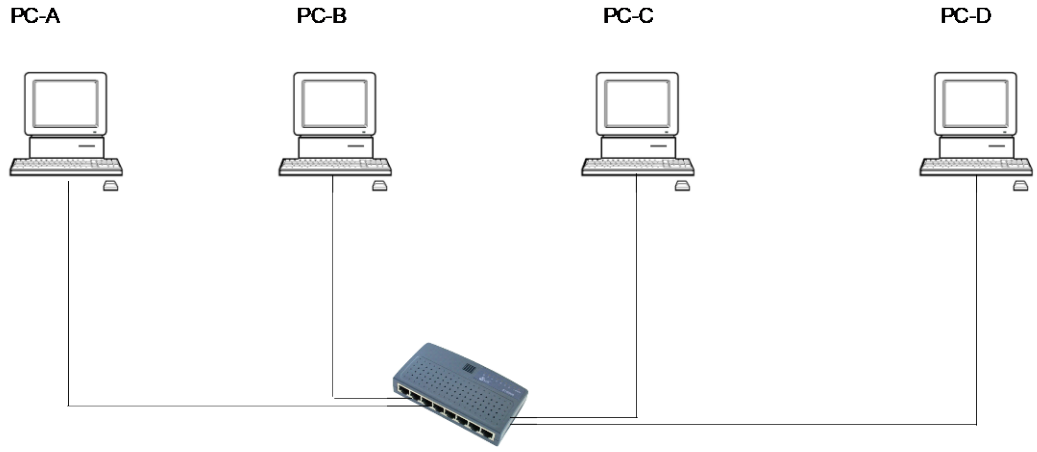
6. Was könnte eine Datenstruktur aussehen, welche bei einem "4-Gewinnt" Multi-Player-Spiel den auf allen Peers replizierten SPIELZUSTAND zu einem gegebenen Zeitpunkt wiedergibt?

```
players{
  0={
    'name' = 'Player1'
    'port' = 9999
    'color' = '#ffffff'
    'x' = 8
    'y' = 5
    'trail' = [[0, 1], [0, 2], [1, 2]]
    'moves' = 3
    'alive' = True
    'timestamp' = '14:05:32'
    'ip_adr' = '127.0.0.1'
  }
}
```

2 Praktikum

2.1 Netzwerkinfrastruktur

Wie bei den Versuchen zu Computernetzen verwendet jede Gruppe eine Insel mit 4 Rechnern, die über einen Sternkoppler zu einem Subnetz zusammengeschlossen sind.



2.2 Gruppenbildung

Sie arbeiten in Gruppen zu je 3 Studierenden zusammen, welche eine gemeinsame verteilte Anwendung entwickeln. JEDER der 3 Teilnehmer muss dabei eine eigene Anwendungskomponente (= Peer) entwickeln.

Obwohl jeder seine Komponente für sich entwickelt, ist vorher Gruppenarbeit notwendig:

1. Die Gruppe diskutiert zentrale Entwurfsentscheidungen.
2. Die Gruppe definiert gemeinsam die Schnittstellen zwischen den Peers.

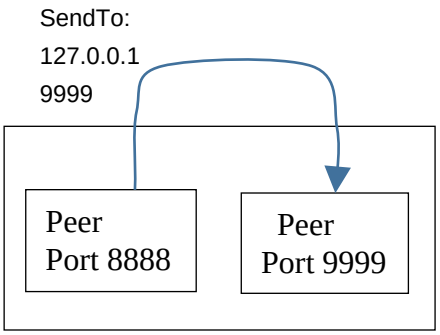
   Checkpoint: Hier holen Sie den Laborleiter, bevor Sie weitermachen und erklären ihre Schnittstelle !

3. Jeder implementiert seine Komponente.
4. Die Gruppe führt das Testen des Verteilten Systems gemeinsam durch.

Hinweise für die Arbeit zuhause:

Für die Versuchsdurchführung zuhause gelten folgende Ergänzungen und Anmerkungen:

- Zuhause haben Sie vermutlich nicht mehrere Rechner zur Verfügung. Sie können hier die einzelnen Peers auf dem GLEICHEN Rechner starten. Dies ist möglich, solange alle Anwendungskomponenten einen ANDEREN Port verwenden, an den sie sich binden.
- Sie kennen vermutlich die IP-Adresse ihres Rechners zuhause nicht. Dies ist aber kein Problem, da Sie die Loopback-IP-Adresse 127.0.0.1 verwenden können.



- Für Remote Tests mit ihren Kommilitonen müssen Sie sich alle von der VM aus in das VPN-Netz der Hochschule einwählen.
- Teilen Sie ihren Kommilitonen mit, auf welcher IP-Adresse und auf welchem Port ihr Peer lauscht. (IP-Adresse muss 141.79.-Präfix haben)
- Versuchen Sie zunächst, ob ein Ping zu den anderen Gruppenmitgliedern funktioniert.
- Schalten Sie nun für ihren jeweiligen Serverport (z.B. 9999) noch die Firewall frei: sudo firewall-cmd --zone=public --add-port=9999/udp
- Natürlich müssen Sie auch die neuen Partner-Adressen in ihrer Partner-Liste ändern.
- Nun sollten Sie in der Lage sein, jeder mit seinem Peer, einen gemeinsamen Chat durchzuführen.

2.3 Entwicklungsumgebung

Die Wahl der Programmiersprache ist für diesen Versuch freigestellt. Die Versuchsanleitung ist auf **Java** ausgerichtet, aber es steht Ihnen frei auch Python, C++ , C# oder C, auf Nachfrage auch eine andere Programmiersprache zu wählen.

Entwicklungsumgebungen für Java und Python sind bereits vorhanden.

Hinweis: Bei Nicht-Java-Implementierungen ist es Bestandteil der Laboraufgabe, ggf. fehlende Teile der Entwicklungsumgebung zu ergänzen.

Für die Kommunikation soll die UDP-Socket-Schnittstelle verwendet werden.

Für Java sind u.a. die folgenden Klassen notwendig:

Funktion	**Package**
Netzwerkkommunikation (Socket-Schnittstelle)	java.net.DatagramSocket java.net.DatagramPacket java.net.InetAddress
Lesen von Konsole	java.util.Scanner
JSON	org.json
Timer	java.util.Timer java.util.TimerTask

2.4 Aufgabenstellung

2.4.1 Teil 1 (1. Labortermin)

Realisieren Sie ein **P2P-basiertes, verteiltes Online-Spiel** mit folgendem Lastenheft:

Anforderungen an das Spiel selbst:

1. Das Spiel basiert auf einem quadratischen Spielfeld der Größe M x M
2. Jeder Spieler hat eine andere Farbe und eine von 4 möglichen Startpositionen (oben Mitte, unten Mitte, links Mitte, rechts Mitte).
3. Durch das Betätigen der Pfeiltasten (↑, ↓, ←, →) bewegt sich ein Spieler um genau eine Rasterzelle in die gewählte Richtung.
4. Jeder Spieler zieht im Verlauf des Spiels eine SPUR von dauerhaft markierten Spielfelder seiner Farbe auf dem Spielfeld hinter sich her. Diese Spur (genau wie die Spielfeldränder) fungieren während des restlichen Spiels als undurchdringbare Wände.
5. Eine Kollision mit dem Spielrand oder einer existierenden Spur (auch der eigenen) führt zum Ausscheiden.
6. Eine Inaktivität von T Sekunden (z.B. T=3) führt zum Ausscheiden.
7. Ziel des Spiels ist es, möglichst lange im Spiel zu bleiben und nicht zu kollidieren.
8. Derjenige Spieler gewinnt, der am Ende noch im Spiel ist. Der Zeitpunkt des Ausscheidens bestimmt die Rangfolge.
9. Während des Spiels wird ein „Spielstand“ angezeigt: Wer noch im Spiel ist, wieviel Züge jeder Spieler bisher gemacht hat, wie die Rangfolge ist.
10. Am Ende wird der Sieger (bzw. ggf. ein Unentschieden) angezeigt.

Anforderungen an die Implementierung:

11. Die Spielfeldgröße M soll konfigurierbar sein (z.B. 25)
12. Die Anwendung soll bis zu N (2 <= N <= 4) Spieler unterstützen.
13. Jeder Peer wird manuell über Konsolkommando gestartet und erfragt über User-Input:
    a) seinen Nickname
    b) seine gewünschte Farbe
    c) seine Port-Nummer
14. Ein Join mit diesen Daten wird abgelehnt, wenn sie mit anderen Spielern kollidieren.
15. Die Menge möglicher Peers (IP-Adressen und Ports) sei bekannt (können als Konstanten im Code realisiert werden), aber Peers kommen via Join nacheinander dazu.
16. Die Startposition eines hinzukommenden Peers soll zufällig aus den noch freien möglichen Startpositionen gewählt werden. Mit Vorrang sollen mittige Startpositionen des Spielfeldrandes gewählt werden.

17. Alle Spielzustände (insbesondere das Spielfeld und der Spielstand) werden in Echtzeit zwischen den Peers synchronisiert und auf jedem Peer graphisch in einem GUI-Fenster visualisiert.
18. Es soll eine "Pure" Peer-2-Peer-Architektur implementiert werden. Ein neuer Peer soll über einen "Friend"-Ansatz integriert werden. Das Kommando zur Integration sei "Join" (mit IP:Port eines Friends)
19. Die Kommunikation soll über UDP erfolgen.
20. Für die Nachrichteninhalte soll JSON verwendet werden.

Folgende Vereinfachungen können gemacht werden:

- Jeder Peer kennt alle anderen Peers direkt, d.h. es ist kein Peer-2-Peer-Routing notwendig.
- Es ist in Ordnung auch Nachrichten an Peers zu schicken, die gar nicht Online sind. D.h. die Gruppenmitgliedschaft muss im ersten Teil der Laboraufgabe NICHT verwaltet werden.
- Der Zustand muss nicht persistent gehalten werden, d.h. alle Zustandsdaten können in Main Memory verwaltet werden.
- Es kann von NICHT gleichzeitig auftretenden Eingaben ausgegangen werden, d.h. es muss keine Vorsorge für Inkonsistenz durch Nebenläufigkeit (Race Conditions) getroffen werden (siehe Option 2*).

Optionale Zusatz-Anforderung 1:

Die Anforderung 6 („Eine Inaktivität von T Sekunden (z.B. T=3) führt zum Ausscheiden.“) kann auch ersetzt werden durch

6‘. Bei Inaktivität von T Sekunden (z.B. T=1) wird die Spur eines Mitspielers vom System automatisch um 1 Position in der zuletzt gewählten Richtung weiterbewegt.

Dies gibt dem Spiel etwas mehr „Dynamik“.

Die Entscheidung ob Anforderung 6 oder Anforderung 6‘ implementiert wird, muss von der ganzen Gruppe zusammen gewählt werden. Möglich ist auch (für einfacheres Debugging) erst Anforderung 6 zu implementieren und testen, danach Anforderung 6‘ als Erweiterung.

Optionale Zusatz-Anforderung 2*:

Wenn Eingaben gleichzeitig erfolgen, dann können die Spielzustände inkonsistent werden. In einem LAN ist dies bedingt durch die kurzen Latenzen relativ unwahrscheinlich, in einem LAN (d.h. als Internet-Spiel) müssten Sie dies zusätzlich berücksichtigen. Deswegen als optionale Anforderung:

21. Stellen Sie sicher, dass Replikate nicht durch fast zeitgleiche Eingaben (siehe *) inkonsistent werden können.

Entwickeln Sie zunächst **zusammen** ein Protokoll zur Realisierung dieser verteilten Anwendung:

- Welche Nachrichtentypen brauchen Sie und was ist für jeden Typ dessen Rolle.

- Wie sind die Nachrichten jedes Nachrichtentyps aufgebaut? Definieren Sie das Nachrichtenformat (welche Struktur, welcher Inhalt)! Dokumentieren Sie das Nachrichtenformat.

  Hinweis: Wer will, kann hierfür eine Schemasprache wie "JSON Schema" verwenden. Ansonsten reichen JSON-Beispielinstanzen.

- Zeichnen Sie Sequenzdiagramme zu den Nachrichten. Welche Eingabe triggert welche Folge von Nachrichten (jeweiliger Typ).

Dokumentieren Sie ihr Design auf den nächsten 2 Seiten !

Sobald Sie fertig sind, lassen Sie ihr Design vom Laborleiter abnehmen.

Hinweise zum Einsatz von LLMs für die Implementierung:

- In dieser Aufgabe kann ein LLM eingesetzt werden:
  1. zur Implementierung des graphischen User Interfaces
  2. für das Debugging

- Die Kommunikation und der verteilte Algorithmus müssen selbst programmiert werden.

- Dies wird kontrolliert, d.h.:
  1. Sie müssen im Labor in der Lage sein, Ihren Nicht-GUI-Code zu erklären
  2. Es wird im Labortest keine Fragen/Aufgaben zum GUI geben
     aber Sie müssen im Labortest in der Lage sein Teile ihres NICHT-GUI-Codes wiederzugeben (Code-Snippets, Struktur-Diagramme, Datenstrukturen,..).

Skizze der Lösung (Teil 1)

1. (Nachrichten-)Typen und Aufbau (ggf. Hilfspapier verwenden):

2. Sequenzdiagramme (ggf. Hilfspapier verwenden)

Input



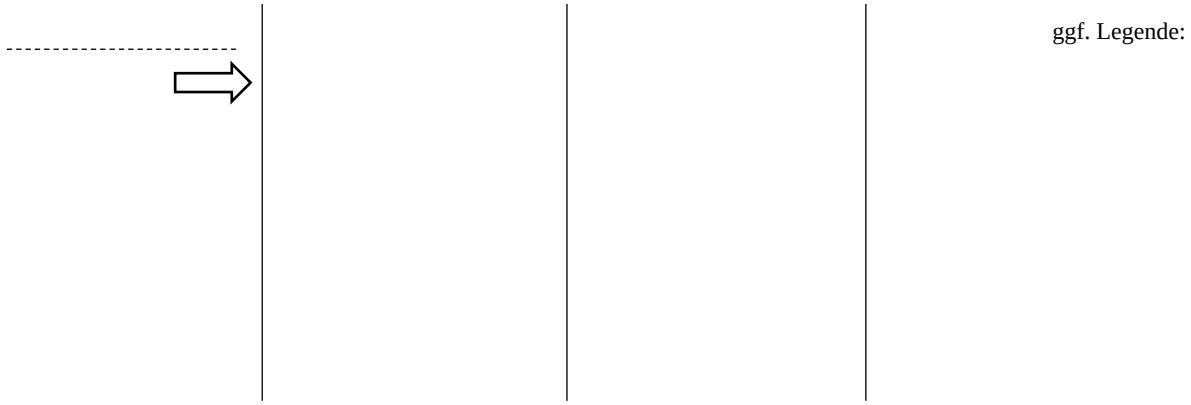
ggf. Legende:

Input



ggf. Legende:

Input



ggf. Legende:

2.4.2 Teil 2 (2. Labortermin)

Am 2. Labortermin sollen Sie ihre (funktionsfähige und bereits lokal getestete) Peer-2-Peer-Anwendung mit den anderen Gruppenmitgliedern **testen**. Jeder der Teilnehmer repräsentiert mit seiner Software einen der Peers.

Um die Firewall zwische n den PCs frei zu schalten, müssen Sie noch den gewählten Port (z.B. 9999) freischalten. Das Kommando hierfür ist:

sudo firewall-cmd --add-port=9999/udp

Falls nötig, korrigieren einzelne Mitglieder danach noch ihre Anwendung.

Holen Sie den Laborleiter sobald Sie hinreichend weit sind (spätestens nach der 1. Doppelstunde) und lassen Sie sich Teil 1 abnehmen !

Danach soll die Anwendung erweitert werden,

indem die Anforderung 4 verallgemeinert wird.

4.alt … jeder Peer kennt diese Obermenge aller möglichen Peers.

4a. **Die Gruppenzusammensetzung ist dynamisch**; die Obermenge aller möglichen Peers ist unbekannt. Es können jederzeit neue Mitglieder (IP:Port) zur Peer-Gruppe dazustoßen oder diese verlassen.

4b. Als Teil der Integration eines neuen Peers, erfährt dieser wer in der Player-Gruppe ist. Es werden NUR Nachrichten an Peers verschickt, welche Online sind.

4c. Neben dem "Join"-Kommando (mit der Adresse eines Friends) gebe es ein "Leave"-Kommando, um die Peer-Gruppe wieder zu verlassen. (Die markierten Felder des Grid für diesen Spieler bleiben bestehen)

4d. Die Teilnehmer, die aktuell Online sind, sollen im Spielstand angezeigt werden.

Zur Umsetzung der Anforderung müssen Sie das Protokoll um ein Peer-2-Peer-Gruppenmanagement erweitern.

Welche zusätzlichen Protokollelemente werden zusätzlich gebraucht?

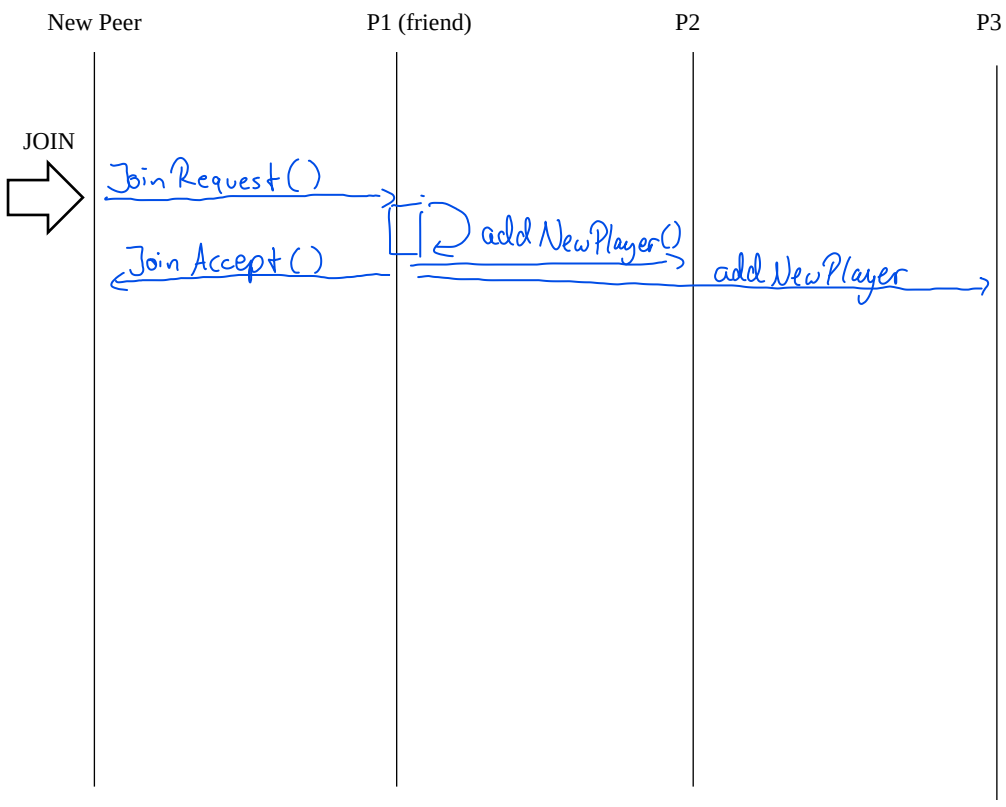
Bitte nächste Seite ausfüllen !

Holen Sie den Laborleiter sobald das Design (nächste Seite) fertig ist.

Skizze der Lösung (Teil 2)

1. **Zusätzliche Nachrichten-Typen und Aufbau:**

2. **Sequenzdiagramm**



2.5 Abgabe (Post-Lab)

Bitte für jede der beiden Teilaufgaben **jeweils 1 ZIP-File** mit allen Inhalten !

Teilaufgabe 1 (späteste Abgabe 14 Tage nach 1. Labortermin):

1. Source Code
2. Executable
3. Lassen Sie ihren eigenen Code (nur Ihren Code, kein Code von Kommilitonen) in 3 Instanzen A, B, C auf einer Maschine laufen (d.h. 3 Screens sichtbar). Machen Sie eine Serie von Screenshots der 3 Screens (immer alle 3 Screens auf 1 Screenshot), welcher einen Spielablauf zeigt, bei dem

   i) Peer A startet und ein paar Eingaben macht.
   ii) Peer B startet und sich dann in Eingaben mit A abwechselt
   iii) Peer C startet und sich in Eingaben mit A und B abwechselt

   Es muss nicht von jedem Zustand ein Screenshot gemacht werden, aber insgesamt sollten in einem PDF ca. 5-10 Screenshots (jeweils von 3 Fenstern) mit erkennbarem Verlauf gesammelt werden bis von den 3 Spieler zwei Spieler ausgeschieden sind. Kommentieren Sie ggf. die Screenshots.

Teilaufgabe 2 (späteste Abgabe 2 Wochen nach Labortermin 2):

1. Source Code
2. Executable
3. Serie von Screenshots wie oben,
   Zusätzlich mache Peer A zwischendurch ein "Leave".

   Screenshots sollen die neue Information mit anzeigen.